University of Colombo School of Computing

SCS 3311 — Mobile Application Design and Development

Mini-Project

HomeSense

Smart Home Monitoring and Control System

Technical Report

Group Members

Name	Index number	Module
R.L. Weerasinghe	23002204	Synchronisation, simulator and safety worker
W.T. Mahagamage	23001038	Device profiles, control interface and reporting
D.M. Isakya	23000643	Floor representation and grid mapping

August 2026

Contents

1 Introduction

The system comprises an Android client, a web-based hardware simulator and a server-side worker process, all communicating through a Firebase Realtime Database. Users manage multiple floor plans, place devices of differing capabilities onto an abstract grid, and control them from the mobile client. The worker enforces safety policy independently of the client.

This report covers the three areas specified for assessment: the synchronising mechanism, the floor representation, and the simulator operations. Architecture, verification and limitations are addressed within those sections or in the appendices.

1.1 Scope of the implementation

All functional requirements of the specification are implemented. Appendix C maps each requirement to the source file implementing it, the test verifying it, and the point in the demonstration at which it is shown.

2 Synchronising mechanism

2.1 State model

The system distinguishes between three related but separate facts about every controllable point in the house. Rather than storing a single boolean per switch, each slot stores three, as shown in Table 1.

Table 1: The three state fields and the component responsible for each

Field	Meaning	Written by
`desiredState`	The state requested by the user	Mobile client, and the worker for cut-offs and schedules
`reportedState`	The state the hardware confirms	Simulator
`status`	The reconciled value presented in the interface	Worker

A single boolean cannot distinguish between a state that has been requested, a state that is actually in effect, and the system's confidence in that information. Three fields make the distinction explicit, and this is what allows the `ERROR` and `DISCONNECTED` states to carry meaning:

- `ERROR` is raised only when `desiredState` and `reportedState` disagree for longer than a tolerance window of ten seconds. A brief disagreement is expected during normal operation, as it represents a command in transit.
- `DISCONNECTED` is raised only when a device's heartbeat is older than fifteen seconds. It takes precedence over all other states. Where no report has been received, presenting `OFF` would express an assumption as though it were an observation.

2.2 Choice of database

Firebase Realtime Database was selected in preference to Cloud Firestore for two reasons.

First, Realtime Database supports per-child listeners. Toggling a single slot produces one `onChildChanged` callback carrying that device, rather than a retransmission of the whole `/devices` subtree. For a grid of independently toggled slots this materially affects both latency and data volume.

Second, Realtime Database provides `onDisconnect()`, which allows the server to record a client's disappearance. Without it, absence would have to be inferred by each observer independently.

Firestore's document-level listener granularity and higher write latency are less suited to this access pattern. The cost of the choice is the absence of meaningful query support and a denormalised tree that must be kept consistent by convention. This is acceptable here because the tree is small, its shape is fixed, and it is documented in full in the project's data contract.

2.3 Propagation of a state change

Figure 1 traces a single toggle from the user's tap to the confirmed status.

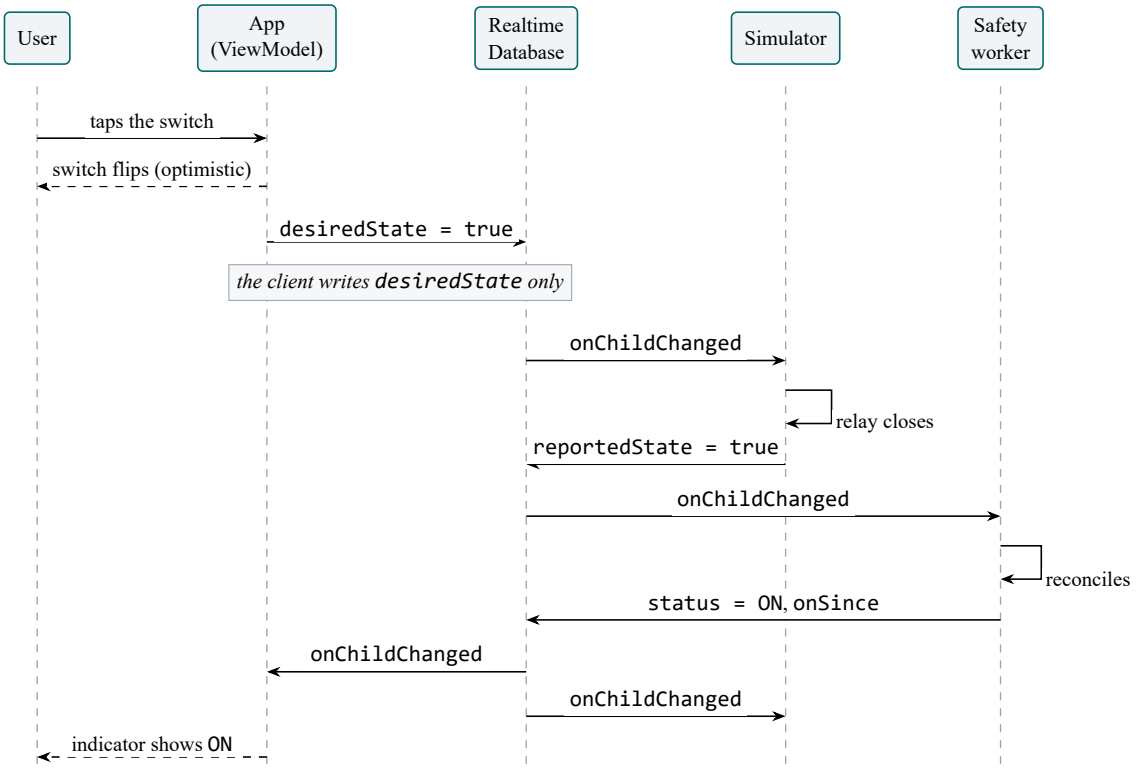

Figure 1: Propagation of a single toggle. The client requests a state; the simulator confirms it; the worker publishes the reconciled status. The indicator reflects confirmation rather than intent.

Three properties follow from this arrangement.

2.3.1 Updates require no manual refresh

The repository interface exposes no `refresh()`, `reload()` or `fetch()` method, because none is required. The device list is a `StateFlow` supplied directly by database child listeners, so a change originating in the simulator, the worker or a second client arrives as a callback and causes recomposition.

The repository tests demonstrate this by emitting a change on the source and asserting that the flow carries it, without any call into the repository.

2.3.2 Toggles are optimistic but reconciled

The switch position changes immediately, since waiting for a network round trip degrades the perceived responsiveness of the interface. However, the client writes only `desiredState` and cannot itself operate a relay. The optimistic position is therefore held for a maximum of four seconds; if `reportedState` has not followed within that window, the control reverts and the user is informed.

Four seconds was chosen to sit between a normal round trip and the worker's ten-second `ERROR` threshold, so that the user is notified shortly before the status indicator changes.

2.3.3 The write separation is enforced by the database

In Realtime Database a `.write` grant propagates to all descendants of the node at which it is declared and cannot be withdrawn at a lower level, so a nested `.write: false` has no effect. Validation rules, by contrast, do restrict client writes, and the Admin SDK bypasses validation entirely.

The security rules use this asymmetry. The fields `status`, `link`, `onSince` and `mismatchSince` carry validators that accept a write only when the value is unchanged:

```
"status": {
  ".validate": "(newData.val() == 'ON'    || newData.val() == 'OFF' ||
                 newData.val() == 'ERROR' || newData.val() == 'DISCONNECTED')
                && ((!data.exists() && newData.val() == 'DISCONNECTED')
                    || newData.val() == data.val())"
}
```

Listing 1: Immutability validator applied to the status field in `database.rules.json`

> An authenticated client cannot publish a status value, clear an `ERROR`, or suppress a `DISCONNECTED` state, while the worker writes these fields without restriction. A limitation of this approach is documented in Appendix D.

2.4 Authentication and tenancy

Synchronisation is only meaningful once it is established whose data is being synchronised. Two questions are answered separately.

Identity is handled by Firebase Authentication with email and password. An anonymous session is also offered so the application can be tried before registering; it is upgraded in place with `linkWithCredential`, which preserves the account identifier, so a guest who later registers keeps the household, history and alerts accumulated in the meantime.

Tenancy is separate. An account belongs to zero or more households, and a household records its members at `homes/$homeId/members`. Every read and write rule consults that record, so a household is confined to the people admitted to it. Members are admitted in one of two ways: by creating the household, which makes the creator its owner, or by redeeming an invite code.

The invite is verified by the database rather than trusted from the client. The joining member writes the code into its own membership record, and the rule accepts the write only if that code exists under `/invites`, names this household, and has not expired. There is no read rule on the invite collection itself, only on an individual code, so a code can be redeemed when already known but the collection cannot be

enumerated.

Two roles are distinguished. Any member may operate devices and edit ordinary schedules; only the owner may add or remove devices, configure `max_on_duration`, and manage membership. Safety configuration is reserved to the owner because altering a cut-off changes the protection applied to everyone in the house.

> Three consequences of introducing sessions had to be handled explicitly rather than emerging on their own. The data layer subscribes through the active household as a flow, so signing out tears down every listener rather than leaving them attached to a household the session is no longer entitled to. The local cache is cleared when the account changes, since Room holds usage history and alerts that would otherwise be visible to the next person to sign in on the same device. And push notifications are published to a topic named after the household rather than to a single shared topic, which would have delivered every family's cut-off alerts to every installation.

2.5 Offline behaviour and usage logging

Realtime Database disk persistence is enabled, so the most recently received tree is available locally and the first frame after a cold start displays real data rather than a loading indicator. A Room database mirrors floors, device layout, alerts and the usage log. Live device status is deliberately excluded from this mirror: an outdated `ON` indicator is more misleading than an honest `DISCONNECTED`.

Usage events are append-only. Each is written at the moment of transition by whichever component caused it, and usage is never reconstructed by comparing successive snapshots. Reconstruction by comparison would omit every transition occurring while the client was closed, which is precisely the period the safety worker exists to cover.

3 Floor representation

3.1 Coordinate model

A device represents one physical node, occupying one grid cell with one network connection and one heartbeat. Its position is stored as integer cell coordinates rather than pixel offsets, so that a plan renders consistently across screen sizes and orientations.

Translation between cell coordinates and screen coordinates is confined to a single class, `GridMapper`, which has no Android dependencies and is therefore testable as a plain JVM unit. Sixteen tests cover the cell-to-pixel round trip for every cell, taps falling on the letterbox margins, taps on cell boundaries and plan edges, degenerate zero-sized canvases, and the property that a given cell resolves identically under phone and tablet geometry.

Because a plan has a fixed aspect ratio and the available canvas generally does not, the plan is fitted within the canvas with its ratio preserved, leaving margins on two sides, as illustrated in Figure 2. All cell arithmetic is performed relative to the fitted rectangle rather than the raw canvas bounds; otherwise the grid would stretch with the window and device positions would drift.

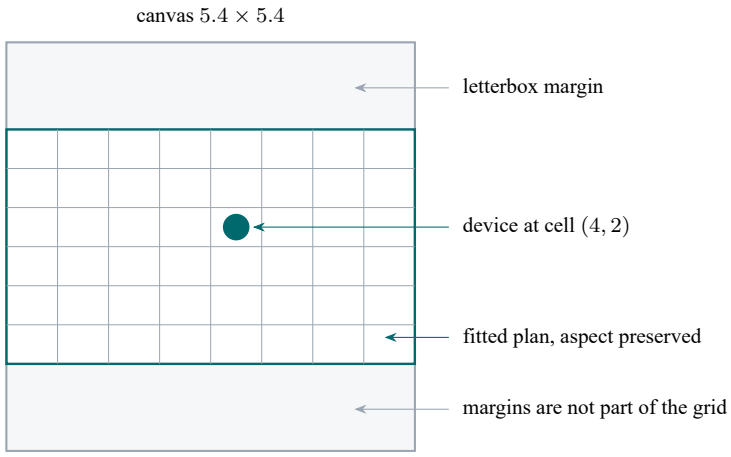

Figure 2: A 4:3 plan fitted into a square canvas. Cell arithmetic is performed relative to the fitted rectangle, so a tap on the margin resolves to no cell rather than to an out-of-range coordinate.

3.2 Representation of plans

The specification calls for an abstract and simple mapping. Plans are accordingly declared as geometry — rooms, doorways and windows in a unit-less coordinate space, defined in `FloorPlanSpec` — and rendered using the Compose `Canvas` API.

This approach has three advantages over bundled raster images. Rendering remains sharp at any screen density, with no bitmap scaling. The aspect ratio is exact, which makes the fitting calculation exact. And since the plans are original work, no third-party image licence needs to be tracked or attributed.

Three sample plans are supplied (ground floor, first floor and an annex), together with a blank grid. A floor record stores only the identifier of its plan, so changing a floor's layout does not affect the devices placed on it.

3.3 Visual encoding

Device markers encode kind through shape and status through fill pattern, as listed in Table 2.

Table 2: Visual encoding of device kind and operational status

Attribute	Encoding
Outlet	Circle
Multi-switch	Square, with one indicator per addressable slot
Camera	Body-and-lens outline
`ON`	Solid fill
`OFF`	Plain outline
`ERROR`	Outline with a cross
`DISCONNECTED`	Dashed outline

No state is conveyed by colour alone anywhere in the application. Status is always presented as colour together with an icon and a text label. This is required for accessibility, and it also ensures the interface remains legible when projected.

Where a multi-switch unit has slots in differing states, its marker shows the most severe of them, so that a fault on one channel is not concealed by two functioning channels.

3.4 Multi-switch units

The specification requires that a multi-switch unit be mapped to a single entity. The data model enforces this: `Device` holds a list of `Slot` values, so a five-gang plate is one device identifier, one grid cell and one heartbeat, with five independently addressable slots and a master control that addresses each in sequence. No code path produces more than one device from a single unit.

```
data class Device(
    val id: String = "",
    val gridX: Int = 0,
    val gridY: Int = 0,
    val kind: DeviceKind = DeviceKind.OUTLET,
    val slots: List<Slot> = emptyList(),   // 1 for an outlet, 2..6 for a gang box
)
```

Listing 2: The device and slot relationship, abridged from `domain/model/Device.kt`

A related modelling decision concerns where scheduling and safety limits belong. Both are properties of a slot rather than of a device kind. An iron is connected to an ordinary socket, and a scheduled lamp may be wired to one channel of a gang box. Modelling `HAZARD` and `LIGHT` as device kinds would make neither arrangement expressible, and the specification's own phrasing, "specialised slots assigned to appliances", indicates the same structure.

The field retains the spelling `max_on_duration` used in the specification. The conversion to an idiomatic Kotlin property name occurs at a single point in the data layer.

4 Simulator operations

4.1 Structure

The simulator is a single self-contained HTML page using the Firebase Web SDK from a content delivery network. It requires no build step and no server. It represents the physical appliances and participates in the protocol rather than merely displaying it.

Devices are rendered as cards grouped by floor. Lamps illuminate when their slot is on, slots with an armed cut-off display a bar filling toward `max_on_duration`, and cameras display a placeholder stream.

4.2 Write responsibilities

The simulator writes exactly two fields: `reportedState`, when its relay changes, and `lastSeen`, at five-second intervals. It does not write `status`. Hardware reports its condition; it does not determine how that condition is classified. The same restriction applies to the mobile client, for the same reason, and is enforced by the same rules file.

The simulator also registers an `onDisconnect()` handler that sets `lastSeen` to zero. Closing the browser tab therefore causes the node to be marked `DISCONNECTED` within the worker's fifteen-second window, with the absence recorded by the server rather than inferred by an observer.

4.3 Fault injection controls

Three controls are provided, each corresponding to a specific requirement.

Table 3: Fault injection controls and the behaviour each demonstrates

Control	Behaviour	Demonstrates
Simulate fault	The relay ceases to act on commands	`desiredState` and `reportedState` diverge and remain divergent; after ten seconds the worker publishes `ERROR`
Unplug	The heartbeat stops	`lastSeen` becomes stale and the worker publishes `DISCONNECTED`
Physical toggle	A change originating at the hardware	The change appears in the client without a refresh, recorded with `source: SIMULATOR`

The fault case is the most informative of the three. The simulator does not write an error state; it simply stops responding. The error is derived by a third process from a disagreement between two others, which is a stronger property than an application setting an indicator on itself.

4.4 Safety worker

The cut-off is implemented in a Node and TypeScript process using the `firebase-admin` SDK, and not in the mobile client. The requirement is that the cut-off protects against a hazard when the phone is unavailable.

Cloud Functions would require the Blaze billing plan and a registered payment method, so the worker is implemented as a standalone process that can be run locally or on a free hosting tier. Each rule is written as a pure function with an injected clock, and a single module performs database access. The rules would therefore transfer to a Cloud Function without modification.

Figure 3 shows the resulting arrangement of components and the fields each is permitted to write.

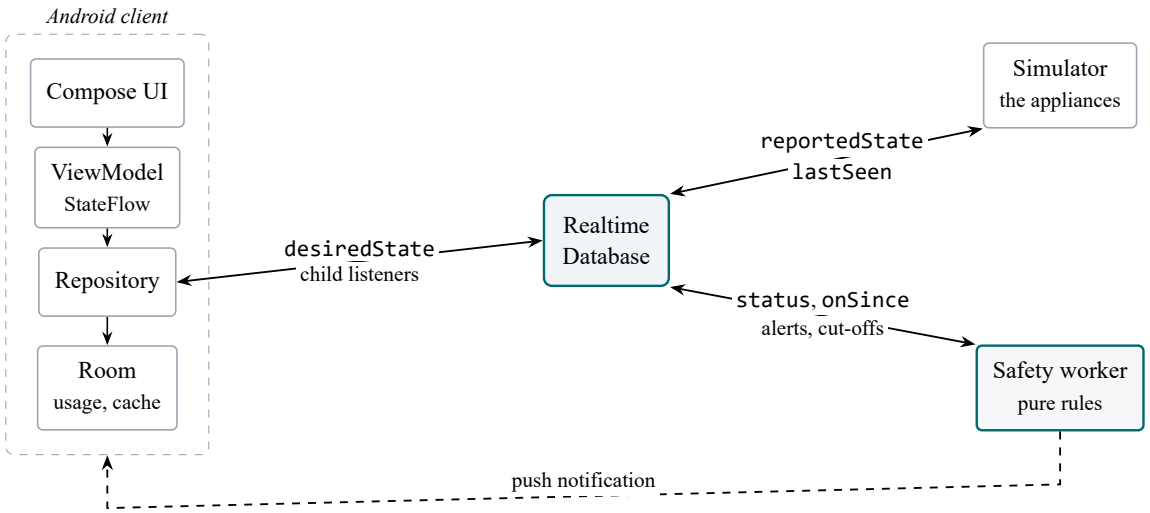

Figure 3: Component arrangement. Each process writes a disjoint set of fields, and the database security rules enforce the separation.

Four rules are evaluated against every device, as summarised in Table 4.

Table 4: Rules evaluated by the safety worker

Rule	Function
`maxOnDurationRule`	Sets `desiredState` to false, appends a `CUTOFF` usage event, raises a critical alert and sends a push notification
`statusRule`	Derives `status` and maintains `onSince` and `mismatchSince`
`staleHeartbeatRule`	Marks a device disconnected when its heartbeat exceeds fifteen seconds
`scheduleRule`	Applies on and off transitions at configured minute boundaries

4.5 Persistence of timer state

Timer state is held in the database rather than in process memory. The field `onSince` is stored, and elapsed time is recomputed from it at every evaluation.

```
const elapsedSeconds = Math.floor((clock.now() - slot.onSince) / 1000);
if (elapsedSeconds < limitSeconds) continue;
// Elapsed time is derived from stored state, not from a timer held in memory.
```

Listing 3: The cut-off condition, abridged from `worker/src/rules/maxOnDurationRule.ts`

A timer implemented with `setTimeout` would be lost if the process terminated, and would be lost without any indication that it had been. Because the state is stored, a worker that crashes, is redeployed or loses power resumes correctly: the next evaluation observes an appliance that has been running for forty-seven seconds against a thirty-second limit and switches it off immediately. This scenario is covered by a unit test, as is the requirement that the cut-off fire at exactly `max_on_duration`.

Two triggers drive the same evaluation function. Child listeners provide immediate reaction to changes, and a thirty-second sweep guarantees recovery after any interruption. Since both invoke the identical function, they cannot produce inconsistent results.

4.6 Schedule semantics

The schedule rule acts on transitions rather than on intervals: it applies a change at the configured on-minute and off-minute only. Enforcing the interval continuously would override manual control, so that a user switching a scheduled light off at 19:00 would find it switched on again within a minute. Acting only at boundaries allows a manual override to persist until the next boundary.

Rule application is idempotent, since at a boundary minute a write occurs only when `desiredState` differs from the target value. The sweep therefore cannot apply a transition twice. The consequence of this design is that a boundary occurring entirely while the worker is unavailable is not applied retrospectively.

A Architecture and technology

The application is written in Kotlin using Jetpack Compose with Material 3, following the MVVM pattern: composable functions observe a ViewModel exposing `StateFlow`, which depends on a repository, which depends on the data sources. No composable function accesses Firebase or Room directly. The minimum supported API level is 26 and the target is 35. The build uses the Gradle Kotlin DSL with a version catalogue.

Dependency injection is performed manually. For an application with a single repository, a dependency injection framework would add a further component to be understood and defended without corresponding benefit.

Two product flavours are provided. The `live` flavour communicates with Realtime Database. The `demo` flavour runs against an in-memory implementation that performs the roles of simulator, worker and database, including a functioning safety cut-off, and therefore exercises the real repository, ViewModels and interface without network access.

Room is not required by the specification. It is used for offline availability and because the reporting screen aggregates usage in SQL rather than in application code over a database snapshot.

B Verification

Table 5: Automated test coverage

Suite	Tests	Coverage
Application unit tests	73	Grid mapping, wire format, repository propagation, Room aggregation, CSV escaping, schedule windows
Worker rule tests	40	Cut-off timing under a controlled clock, restart recovery, status precedence, schedule boundaries
Total	**113**	

Both `./gradlew assembleDebug` and `./gradlew test` complete successfully, as do `npm test` and type checking in the worker project. A continuous integration workflow runs both suites on every push from a checkout containing no credentials.

C Requirement traceability

Each requirement is mapped to its implementation, its verification, and the point in the demonstration recording at which it is shown. Requirements marked **M** describe integration behaviour spanning a mobile device, a browser and a server process, and are verified by demonstration rather than by automated test.

#	Requirement	Implementation	Verification	Demo
R1	Multiple floor plans, added and managed	`ui/floors/`	Repository tests	4:30

#	Requirement	Implementation	Verification	Demo
R2	Abstract grid mapping over layouts	`ui/plan/GridMapper.kt`	`GridMapperTest`, 16 tests	2:15
R3	Sample plans supplied	`FloorPlanSpec.kt`	Compose preview	2:15
R4	Toggling reactively updates the interface	`ui/device/`, `DeviceViewModel`	Repository tests	6:00
R5	Four operational states displayed	`Slot.kt`, `StatusPill.kt`	`SlotTest`, worker tests, previews	2:45
R6	Outlets as single-node binaries	`DeviceKind.OUTLET`	`WireTest`	6:00
R7	Multi-switch as a single entity	`Device.kt`	`SlotTest`, repository tests	5:00
R8	`max_on_duration` on hazard slots	`maxOnDurationRule.ts`	Worker tests, exact timing	8:30
R9	Lights on preset time periods	`scheduleRule.ts`	`ScheduleTest`, worker tests	7:45
R10	Cameras with simulated streams	`CameraCard.kt`	Compose previews	9:30
R11	Client changes reach the database	`RealtimeDatabaseSource.kt`	Repository tests	6:00
R12	External changes reach the client	`callbackFlow` listeners	Repository tests	11:00
R13	Server-side cut-off	`maxOnDurationRule.ts`	Worker tests, live run	15:30
R14	Cut-off raises an alert	`notifications.ts`, `ui/alerts/`	Demonstration **(M)**	15:30
R15	Usage reporting	`ui/report/`, `UsageDao`	`UsageDaoTest` and others	18:00
R16	Web simulator reflecting updates	`simulator/index.html`	Demonstration **(M)**	10:30
R17	Simulator listens to the database	`simulator/index.html`	Demonstration **(M)**	6:00
D1	Repository and APK	`apk/HomeSense-1.0.apk`	Signature verified	—
D2	Technical report	This document	—	—
D3	Demonstration, three members	Demonstration script	—	0:00

D Limitations

Client roles are not distinguished. The mobile client and the simulator both authenticate anonymously, so the security rules cannot separate them. A modified client could write `reportedState` as though it were

hardware. Separating the two roles would require custom authentication claims and a service to issue them, which is outside the scope of this assignment. No client can write `status`, which is the property on which the safety argument depends.

A single home is supported. The database tree is keyed by home identifier, so support for multiple homes would be additive, but it is not implemented.

The worker must be running. If the worker stops, no status is derived, no cut-off occurs and no schedule is applied. This is a single point of failure. The simulator therefore displays the worker's own heartbeat and reports when it is not running.

Camera feeds are simulated, as the specification permits. Frames are generated locally. The configured snapshot and stream URIs are displayed beneath the tile so that the mechanism a real camera would use remains visible.

Energy figures are estimates. They assume the rated wattage is drawn for the whole period during which an appliance is on, and are labelled as estimates wherever they appear.

E Design decisions

Ref	Decision
0001	Realtime Database in preference to Firestore, for per-child listeners and `onDisconnect()`
0002	Android Gradle Plugin 8.13.2 with compileSdk 36, since current androidx releases require AGP 9 and a correspondingly recent Android Studio
0003	A standalone Node worker in preference to Cloud Functions, which require a billing plan; the rules remain portable to Cloud Functions unchanged